Análisis de la arquitectura, conectividad y limitaciones de Windows Subsystem for Linux 2 (WSL2)

Canteros, Leonardo Oscar; Fernandez, Facundo Nicolas; Gauna, Julieta Itatí; Ojeda, Juan Agustín

Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste.

juli.gauna09@gmail.com, leonardocanteros2001@gmail.com, facufer00@gmail.com, oagusj@outlook.com

Resumen. Este trabajo analiza la arquitectura, conectividad y limitaciones de Windows Subsystem for Linux 2 (WSL2) mediante cinco experimentos realizados con Ubuntu 22.04 sobre Windows 11. Se evaluaron el funcionamiento del kernel Linux, la conectividad de red mediante NAT y mirrored, la integración de sistemas de archivos, el acceso a dispositivos físicos y la persistencia de servicios. Los resultados evidenciaron diferencias de rendimiento entre los sistemas de archivos Linux y Windows, la necesidad de configuraciones adicionales para acceder a servicios y dispositivos, y la detención de la distribución pese a mantener Nginx habilitado. Se concluye que WSL2 proporciona un entorno Linux funcional e integrado con Windows, aunque presenta particularidades que deben considerarse en escenarios que requieren autonomía y disponibilidad continua.

Palabras clave: WSL2, virtualización, conectividad de red, sistemas de archivos, persistencia de servicios.

Abstract. This study analyzes the architecture, connectivity, and limitations of Windows Subsystem for Linux 2 (WSL2) through five experiments conducted using Ubuntu 22.04 on Windows 11. The experiments examined the Linux kernel, network connectivity in NAT and mirrored modes, file system integration, physical device access, and service persistence. The results revealed performance differences between Linux and Windows file systems, the need for additional configurations to access services and devices, and the termination of the distribution despite Nginx being enabled. The study concludes that WSL2 provides a functional Linux environment integrated with Windows, although its characteristics must be considered in scenarios requiring system autonomy and continuous availability.

Keywords: WSL2, virtualization, network connectivity, file systems, service persistence.

1. Introducción

En la actualidad, Windows y Linux ocupan un lugar importante en distintos ámbitos informáticos. Según StatCounter GlobalStats, en septiembre de 2026 Windows representó aproximadamente el 76 % de la participación de los sistemas operativos de escritorio a nivel mundial [1]. Por su parte, Linux mantiene una presencia destacada en entornos de servidores y servicios web [2]. Esta distribución de usos pone de manifiesto la utilidad de contar con herramientas que permitan trabajar con ambos sistemas operativos. Ante este panorama, Microsoft desarrolló Windows Subsystem for Linux (WSL), una tecnología que permite ejecutar distribuciones y herramientas Linux directamente dentro de Windows [3].

La primera versión, conocida como WSL1, utilizaba una capa de compatibilidad que traducía las llamadas al sistema de Linux para su ejecución en Windows. Sin embargo, presentaba limitaciones de compatibilidad con determinadas aplicaciones. Para superar estas dificultades, Microsoft anunció WSL2 en 2019, incorporando un kernel Linux real y mejorando la compatibilidad con las aplicaciones y herramientas de este sistema operativo [4].

A diferencia de WSL1, WSL2 utiliza una máquina virtual ligera administrada automáticamente por Windows, que permite ejecutar el kernel Linux sin necesidad de configurar una máquina virtual tradicional. Esta arquitectura facilita la integración entre ambos entornos y permite utilizar herramientas Linux desde Windows, aunque introduce características particulares en la conectividad de red, el acceso a archivos y la administración de recursos [5].

Entre estas características se encuentra la interoperabilidad entre Windows y Linux, que permite ejecutar herramientas de ambos sistemas, acceder a sus archivos y compartir los recursos físicos del equipo anfitrión, como el procesador y la memoria RAM [6], [7]. En cuanto a la conectividad, WSL2 utiliza NAT de manera predeterminada y también admite el modo de red reflejado (*mirrored*). Estos mecanismos permiten establecer la comunicación mediante interfaces virtuales, direccionamiento IP y resolución DNS, así como acceder a servicios Linux desde Windows o desde otros equipos de la red, según la configuración utilizada [8].

Respecto al almacenamiento, WSL2 utiliza un sistema de archivos Linux basado en ext4, alojado en un disco virtual, y permite acceder a las unidades de Windows mediante rutas como /mnt/c [6]. Esta integración facilita el intercambio de archivos entre ambos entornos, aunque presenta particularidades relacionadas con el acceso, los permisos y el rendimiento de las operaciones [6], [7].

A partir de estas características, la presente investigación tiene como objetivo analizar el funcionamiento de WSL2, considerando su arquitectura de virtualización ligera, los mecanismos de integración con Windows y las capacidades y limitaciones técnicas que presenta en distintos ámbitos de funcionamiento. Para ello, se plantearon cinco experimentos orientados a identificar las características de su arquitectura y del kernel Linux, examinar los mecanismos de conectividad de red, analizar la integración entre los sistemas de archivos de Windows y Linux, evaluar el acceso a dispositivos físicos y estudiar las funcionalidades y restricciones relacionadas con la administración del entorno Linux. Mediante estas pruebas, se busca comprender cómo la arquitectura de WSL2 condiciona su funcionamiento y qué operaciones pueden realizarse directamente, cuáles requieren configuraciones adicionales y cuáles presentan restricciones en el entorno estudiado.

2. Instrumentación

Para el desarrollo de las pruebas experimentales se utilizó una computadora equipada con un procesador AMD Ryzen 5 5625U de 6 núcleos y 12 procesadores lógicos, 16 GB de memoria RAM y una unidad de almacenamiento SSD M.2 de 480 GB, con el sistema operativo Windows 11. Sobre este entorno se instaló Windows Subsystem for Linux 2 (WSL2), versión 3.0.3.0, con la distribución Ubuntu 22.04.5 LTS y el kernel Linux 6.18.54.1-microsoft-standard-WSL2.

Las herramientas utilizadas fueron:

- Windows PowerShell para administrar y configurar WSL2.
- Nginx para realizar pruebas de conectividad HTTP.
- Python 3 para procesar archivos y ejecutar diagnósticos.

- usbipd-win para conectar dispositivos USB a WSL2.
- Utilidades de Linux.

Asimismo, se utilizaron comandos como uname, ip, curl, df, lsblk, lsusb, sha256sum, chmod y systemctl, entre otros, para verificar las características del entorno y registrar los resultados experimentales.

El laboratorio se organizó con una computadora principal que ejecutó Windows 11 como sistema anfitrión y Ubuntu mediante WSL2. En esta última se configuró el servidor Nginx. Se empleó una segunda computadora conectada a la misma red local para verificar el acceso HTTP desde un equipo externo.

Las pruebas abarcaron cinco aspectos: arquitectura y kernel Linux, conectividad de red mediante NAT y mirrored, integración y rendimiento de los sistemas de archivos, acceso a dispositivos físicos y persistencia de servicios. Los procedimientos y resultados se documentaron mediante comandos, mediciones y capturas de pantalla.

3. Desarrollo

3.1. Identificación de la arquitectura y del kernel de WSL2

Para identificar las características del entorno WSL2, se ejecutaron los comandos wsl --version, wsl --status y wsl --list --verbose desde Windows PowerShell. Se verificó la versión 3.0.3.0 de WSL y la ejecución de Ubuntu 22.04 bajo WSL2 (Fig. 1).

```
PS C:\WINDOWS\system32> wsl --version
Versión de WSL: 3.0.3.0
Versión de kernel: 6.18.54.1-1
Versión de WSLg: 1.0.79
Versión de MSRDC: 1.2.7391
Versión de Direct3D: 1.611.1-81528511
Versión de DXCore: 10.0.26100.1-240331-1435.ge-release
Versión de Windows: 10.0.26200.9711
PS C:\WINDOWS\system32> wsl --status
Distribución predeterminada: Ubuntu-22.04
Versión predeterminada: 2
WSL1 no es compatible con la configuración actual del equipo.
Habilita el componente opcional "Subsistema de Windows para Linux" para usar WSL1.
PS C:\WINDOWS\system32> wsl --list --verbose
  NAME            STATE           VERSION
* Ubuntu-22.04    Running         2
PS C:\WINDOWS\system32>
```

Fig. 1. Identificación de la versión de WSL y de la distribución instalada.

Posteriormente, mediante cat /etc/os-release, uname -r y cat /proc/version, se identificó Ubuntu 22.04.5 LTS y el kernel Linux 6.18.54.1-microsoft-standard-WSL2 (Fig. 2).

```
redes_2026_grupo27@Julieta09:/mnt/c/WINDOWS/system32$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
redes_2026_grupo27@Julieta09:/mnt/c/WINDOWS/system32$ uname -r
6.18.54.1-microsoft-standard-WSL2
redes_2026_grupo27@Julieta09:/mnt/c/WINDOWS/system32$ cat /proc/version
Linux version 6.18.54.1-microsoft-standard-WSL2 (root@9b80f41eb17c) (gcc (GCC) 13.2.0, GNU ld (GNU Binut
ils) 2.41) #1 SMP PREEMPT_DYNAMIC Tue Oct  6 19:13:07 UTC 2026
redes_2026_grupo27@Julieta09:/mnt/c/WINDOWS/system32$
```

Fig. 2. Identificación de la distribución Ubuntu y del kernel Linux.

Finalmente, los comandos systemd-detect-virt y ps -p 1 -o pid,comm,args permitieron comprobar que el entorno de virtualización corresponde a WSL y que systemd se ejecuta como proceso PID 1 (Fig. 3). Los resultados son consistentes con la arquitectura de WSL2, basada en la ejecución de un kernel Linux real mediante virtualización ligera.

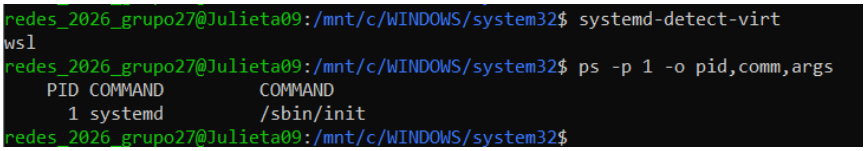


Fig. 3. Verificación del entorno de virtualización y del proceso de inicialización.

3.2. Conectividad entre Windows y WSL2

Para analizar la conectividad de WSL2, se examinó la configuración de red en modo NAT mediante los comandos ip -4 addr e ip route. Se identificó una dirección IPv4 privada y una puerta de enlace virtual (Fig. 4). Asimismo, se verificó que el servidor Nginx respondía correctamente a las solicitudes HTTP realizadas desde Ubuntu (Fig. 5).

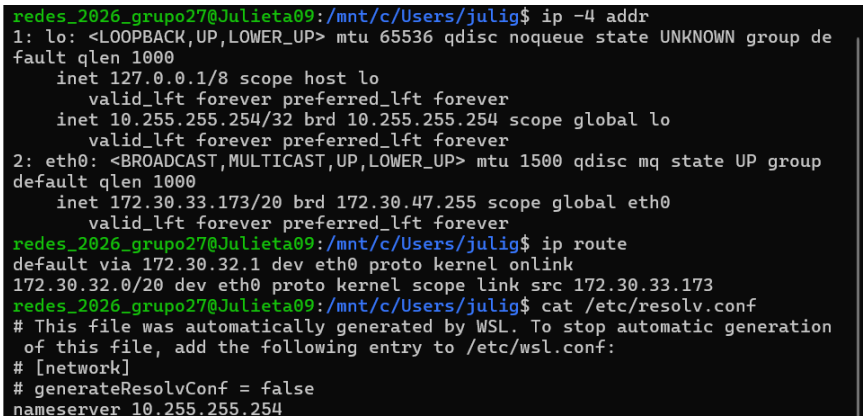


Fig. 4. Dirección IPv4, rutas y resolución de nombres de WSL2 en modo NAT.

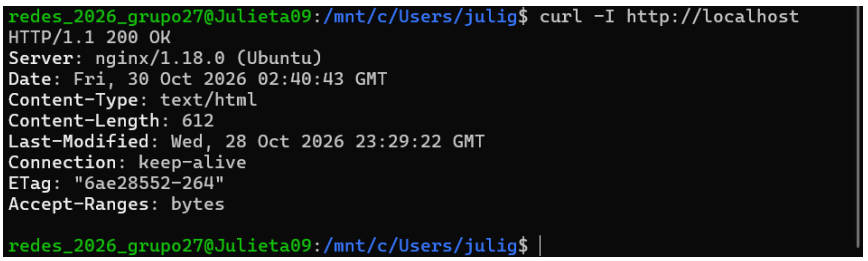


Fig. 5. Respuesta HTTP 200 OK del servidor Nginx mediante localhost.

Al intentar acceder al servidor desde una segunda computadora de la red local, la conexión no se estableció. Posteriormente, se comprobó que era posible habilitar el acceso en modo NAT mediante una redirección TCP con portproxy y una regla de entrada en el firewall de Windows. Esta configuración se menciona sin incorporar capturas adicionales.

A continuación, se modificó el archivo .wslconfig para habilitar el modo de red mirrored (Fig. 6). Tras reiniciar WSL2, se verificó que Ubuntu utilizaba la misma dirección IPv4 de la interfaz de red de Windows (Fig. 7).



Fig. 6. Configuración del modo de red mirrored en el archivo .wslconfig.

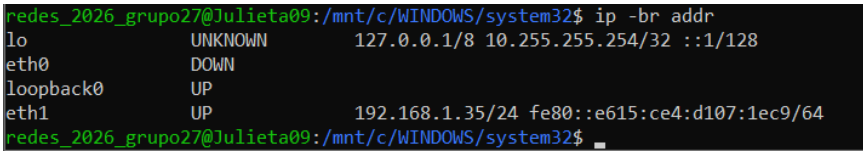


Fig. 7. Verificación de la dirección IPv4 de Ubuntu en modo mirrored.

En la prueba de acceso remoto con mirrored, inicialmente la conexión fue bloqueada. Se identificó la política de entrada del firewall de Hyper-V y se incorporó una regla para permitir el tráfico TCP al puerto 80 (Fig. 8). Con esta configuración, la segunda computadora pudo acceder al servidor Nginx sin utilizar portproxy (Fig. 9).

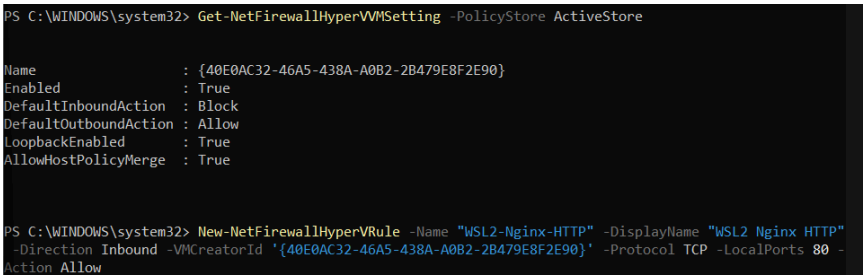


Fig. 8. Política de entrada y creación de la regla HTTP en el firewall de Hyper-V.

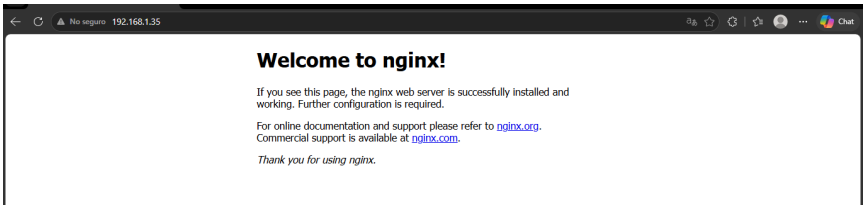


Fig. 9. Acceso exitoso al servidor Nginx desde una segunda computadora de la red local.

3.3. Integración de sistemas de archivos

Se examinó el acceso a archivos entre el sistema de archivos de Ubuntu y las unidades de Windows montadas en /mnt/c. Para ello, se utilizó un archivo CSV de vuelos de enero de 2025, con 539.748 líneas (incluido el encabezado) y un tamaño de 243.177.378 bytes. El archivo se almacenó en /home y se copió a /mnt/c mediante cp, verificando posteriormente su presencia y tamaño (Fig. 10).

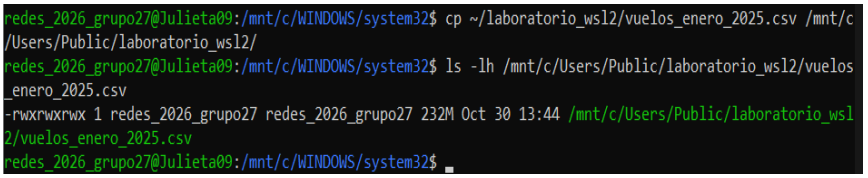


Fig. 10. Copia del conjunto de datos desde el sistema de archivos Linux a la unidad de Windows.

La integridad de ambas copias se verificó con sha256sum: los valores obtenidos coincidieron, lo que confirmó que la transferencia no alteró el contenido del archivo (Fig. 11).

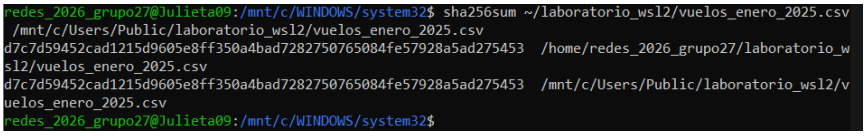


Fig. 11. Comparación de las sumas SHA-256 del archivo en /home y /mnt/c.

Para comparar el comportamiento de lectura, se ejecutó cinco veces el mismo script Python sobre cada ubicación. En todos los casos se identificaron 8.327 vuelos con origen JFK. El tiempo medio de procesamiento fue de 4,66 s en /home y 10,71 s en /mnt/c, aproximadamente 2,3 veces mayor en esta última ubicación (Fig. 12). Estos resultados corresponden al entorno ensayado y no constituyen una comparación general del rendimiento de Windows y Linux.

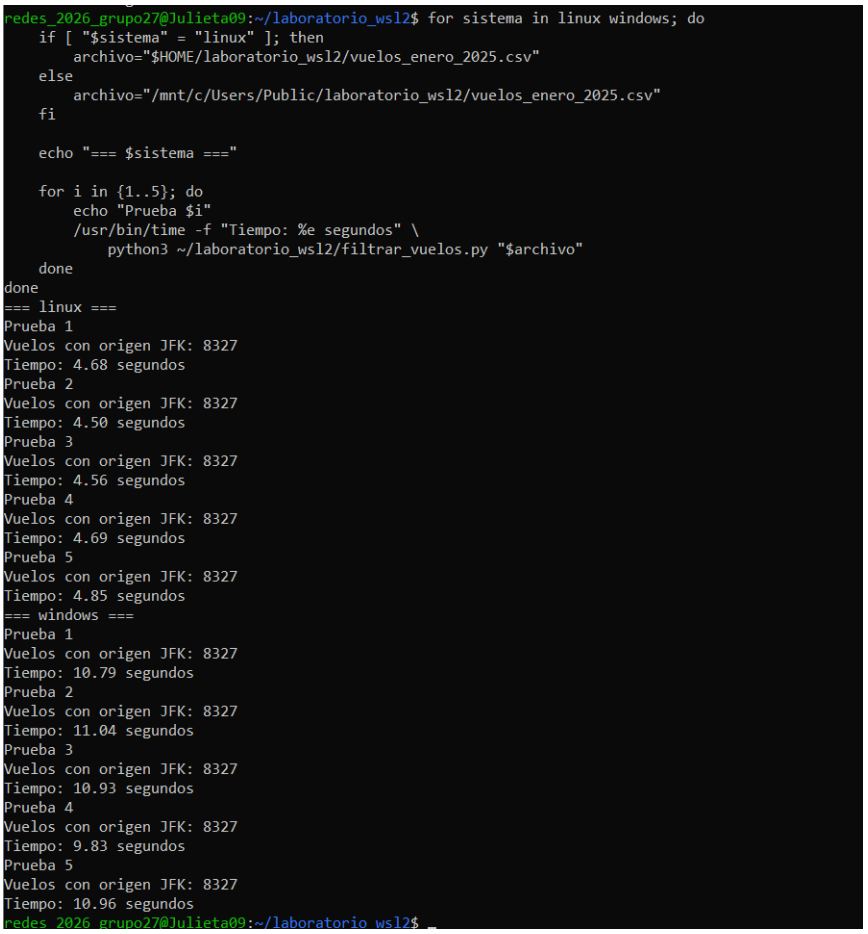


Fig. 12. Cinco ejecuciones del filtrado de vuelos desde /home y /mnt/c.

Por último, se compararon los permisos visibles de los archivos y se aplicó chmod 600 a copias temporales. En /home, los permisos cambiaron a rw-------; en /mnt/c, permanecieron como rwxrwxrwx (Fig. 13). Esta diferencia es consistente con la gestión de permisos de las unidades Windows montadas mediante DrvFs, cuyo comportamiento depende de las opciones de montaje y de los permisos de Windows.

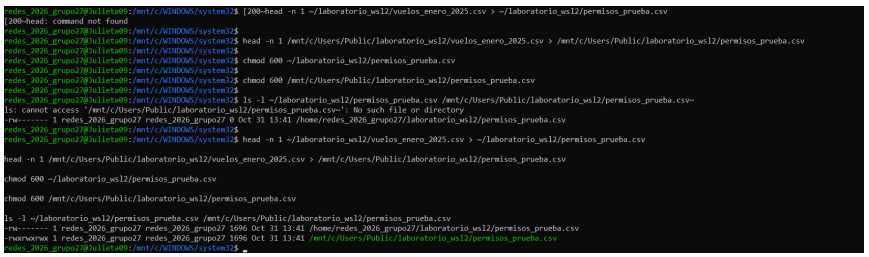


Fig. 13. Verificación de permisos y comportamiento de chmod 600 en ambas ubicaciones.

3.4. Acceso al hardware y dispositivos físicos

Se desarrolló una herramienta de diagnóstico en Python 3 para examinar funcionalidades y restricciones de WSL2 mediante 25 comprobaciones agrupadas en módulos del kernel, virtualización anidada mediante KVM y dispositivos USB.

La selección de estas áreas se fundamentó en reportes de usuarios sobre dificultades con dispositivos loop [12], disponibilidad de KVM en equipos AMD [13] y conexión USB mediante usbipd-win [14]. Estas publicaciones motivaron las pruebas para determinar si dichas condiciones se presentaban en el entorno evaluado.

El programa utiliza subprocess para ejecutar comandos de Linux y consultar recursos del sistema, como /proc/config.gz y /dev/kvm. Clasifica e interpreta los resultados y conserva las evidencias en archivos CSV y de texto.

La herramienta es de solo lectura y no modifica la configuración del sistema. Sus resultados se complementaron con pruebas manuales de funcionamiento de loop, conexión USB y disponibilidad de servicios, permitiendo distinguir entre soporte integrado, recursos no expuestos y funcionalidades que requieren mecanismos adicionales de integración con Windows.

3.4.1. Ejecución de la herramienta de diagnóstico

La herramienta se ejecutó en Ubuntu 22.04 sobre WSL2 mediante python3 diagnostico_wsl2_v6_1.py. La ejecución finalizó con 25 pruebas y registró las evidencias en resultados_wsl2. La Fig. 14 presenta la salida agrupada por temática, con explicaciones directamente en la consola.



Fig. 14. Ejecución del diagnóstico automatizado de módulos, KVM y acceso USB.

3.4.2. Módulos del kernel y soporte loop

La consulta modinfo loop devolvió «Module loop not found». Para determinar si ello implicaba ausencia de soporte, se examinó la configuración del kernel mediante zgrep sobre /proc/config.gz. El resultado CONFIG_BLK_DEV_LOOP=y indicó que la funcionalidad está integrada directamente en el kernel, sin requerir un archivo de módulo independiente. En consecuencia, el programa clasifica el caso como «NO MODULAR» e «INTEGRADO», evitando interpretar el fallo de modinfo como una restricción funcional demostrada.

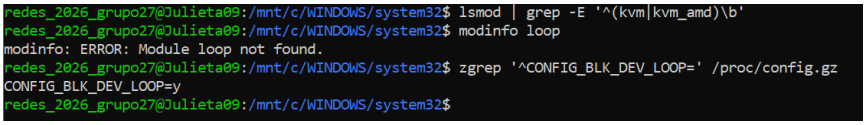


Fig. 15. Consulta manual de loop y verificación de CONFIG_BLK_DEV_LOOP=y.

Para complementar el diagnóstico, se realizó una prueba funcional mediante losetup, asociando un archivo temporal de 16 MB al dispositivo /dev/loop0. Posteriormente, se verificó la asociación y se efectuó una lectura de 1 MB mediante dd, obteniendo el resultado LECTURA_LOOP_OK. Esto confirmó que la funcionalidad loop, además de estar integrada en el kernel, podía utilizarse correctamente para estas operaciones dentro del entorno WSL2 evaluado (Fig. 16).

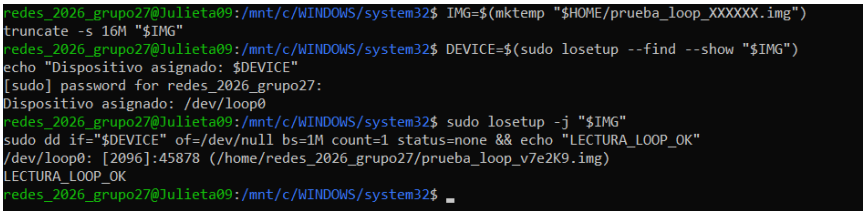


Fig. 16. Asociación de un archivo temporal al dispositivo loop y verificación de lectura exitosa en WSL2.

3.4.3. Virtualización anidada y disponibilidad de KVM

Se examinó la disponibilidad de aceleración KVM. Aunque Windows informó que la virtualización del procesador AMD Ryzen 5 5625U estaba habilitada en el firmware, Ubuntu no presentó el dispositivo /dev/kvm ni expuso la extensión svm en /proc/cpuinfo. Por su parte, la configuración del kernel indicó CONFIG_KVM=m y CONFIG_KVM_AMD=m; modinfo kvm_amd localizó el archivo del módulo, mientras que lsmod no mostró módulos KVM cargados. La simulación modprobe -n -v kvm_amd enumeró irqbypass, kvm y kvm-amd como dependencias y módulos previstos, sin cargarlos.

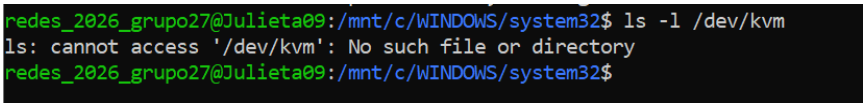


Fig. 17. Ausencia del dispositivo /dev/kvm en Ubuntu sobre WSL2.

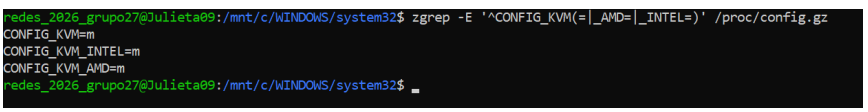


Fig. 18. Configuración modular del soporte KVM en el kernel.

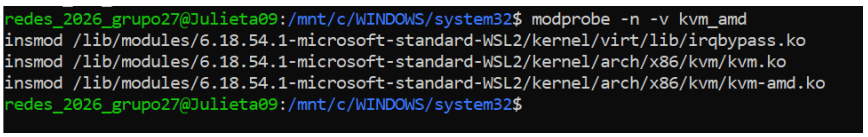


Fig. 19. Simulación no destructiva de la carga de módulos KVM.

Los resultados permiten afirmar que el kernel dispone de soporte modular para KVM, pero que el dispositivo necesario para utilizarlo no estaba expuesto en la instalación examinada. No se intentó cargar el módulo ni se determinó la causa definitiva de esa condición.

3.4.4. Acceso a dispositivos USB físicos

El diagnóstico registró que Windows reconocía el dispositivo de almacenamiento USB con identificador 0930:6545 en estado Shared, mientras que Ubuntu no lo enumeraba en ese momento. Asimismo, lsusb terminó con código de salida 1 sin mensaje de error y no se encontró /dev/bus/usb. Estos resultados describen la visibilidad del dispositivo en el momento de la ejecución; no demuestran por sí solos una incompatibilidad general con USB.



Fig. 20. Dispositivo USB reconocido por Windows y habilitado para compartirse (Shared).

La posibilidad de superar esta condición ya se había comprobado experimentalmente mediante usbipd-win: después de habilitar el dispositivo con usbipd bind --busid 2-2, se utilizó usbipd attach --wsl --busid 2-2. Ubuntu pasó a enumerar el identificador 0930:6545 mediante lsusb. Tras ejecutar usbipd detach --busid 2-2, el dispositivo físico dejó de aparecer. Estas operaciones corresponden al laboratorio previo; la herramienta Python no las ejecuta ni modifica el estado de los dispositivos.

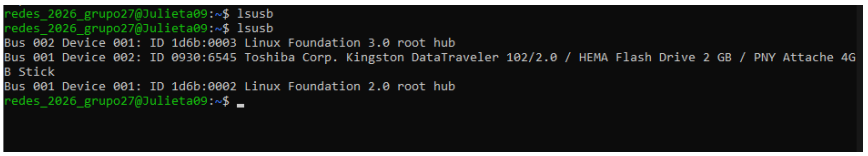


Fig. 21. Reconocimiento del dispositivo USB físico en Ubuntu después de adjuntarlo mediante USB/IP.

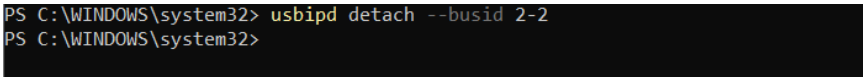


Fig. 22. Desconexión del dispositivo USB físico; permanecen concentradores raíz virtuales.

3.4.5. Persistencia de servicios: comprobación manual

De forma independiente del programa Python, se examinó la persistencia de Nginx administrado por systemd. Tras iniciar Ubuntu, systemctl confirmó que Nginx estaba habilitado (enabled) y activo (active). Al cerrar la última sesión interactiva con exit, una consulta inicial desde PowerShell mostró Ubuntu-22.04 en estado Running y otra,

aproximadamente cuatro segundos después, lo mostró como Stopped. La prueba evidencia que habilitar un servicio mediante systemd no garantiza por sí solo que la distribución WSL2 permanezca ejecutándose. El intervalo observado no constituye un tiempo fijo de detención.

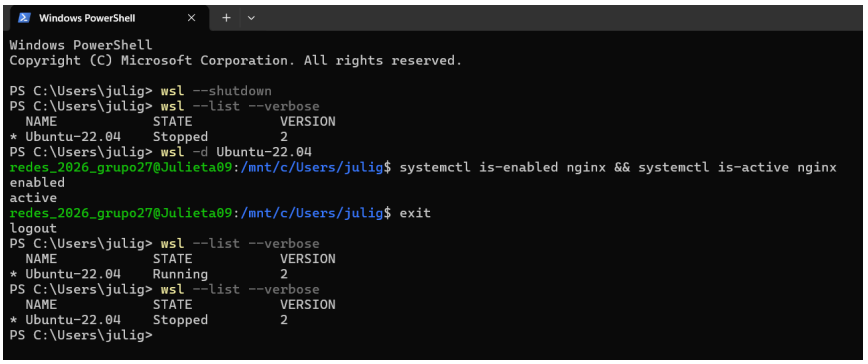


Fig. 23. Comprobación manual de Nginx y transición de Ubuntu de Running a Stopped.

En conjunto, las pruebas diferenciaron tres situaciones: una funcionalidad integrada que no requiere módulo independiente (loop), soporte modular existente sin disponibilidad actual del dispositivo KVM y acceso USB físico que requirió un mecanismo explícito de integración con Windows. La persistencia se examinó por separado porque depende del ciclo de vida de la distribución y no de una consulta aislada a systemd.

4. Resultados y análisis

Los experimentos permitieron comprobar que WSL2 ejecuta un kernel Linux real mediante un entorno de virtualización administrado por Windows. La distribución Ubuntu reconoció el sistema de inicialización systemd y permitió utilizar herramientas habituales de Linux, lo que permitió comprobar el funcionamiento de las herramientas y servicios Linux evaluados. Sin embargo, las pruebas también identificaron diferencias en la conectividad, el acceso a archivos, la disponibilidad de dispositivos físicos y la persistencia de servicios.

En cuanto a la conectividad, el modo NAT permitió la comunicación local con el servidor Nginx, pero el acceso desde otra computadora requirió una redirección de puertos y una regla del firewall de Windows. Con el modo mirrored, Ubuntu utilizó la dirección IPv4 del equipo anfitrión y fue posible acceder al servidor desde la red local sin utilizar portproxy, aunque resultó necesario habilitar el tráfico entrante mediante el firewall de Hyper-V. Estos resultados muestran que la exposición de servicios Linux depende de la configuración de red y de los mecanismos de seguridad de Windows.

Respecto de los sistemas de archivos, se comprobó la integridad de los datos transferidos entre /home y /mnt/c mediante la coincidencia de sus sumas SHA-256. No obstante, el procesamiento del mismo archivo CSV registró un tiempo promedio de 4,66 segundos en /home y 10,71 segundos en /mnt/c, aproximadamente 2,3 veces mayor en esta última ubicación. Asimismo, se observaron diferencias en la aplicación de permisos mediante chmod, relacionadas con el comportamiento de las unidades Windows montadas en WSL2. Estos resultados evidencian que la ubicación de los archivos influye en determinadas operaciones realizadas desde Linux.

Para examinar las funcionalidades y restricciones de WSL2, se desarrolló una herramienta de diagnóstico en Python que ejecutó 25 comprobaciones agrupadas en módulos del kernel, virtualización anidada mediante KVM y acceso a dispositivos USB. El programa permitió identificar componentes configurados y recursos

disponibles, mientras que las pruebas manuales complementarias permitieron verificar determinadas operaciones funcionales.

En el caso de loop, aunque modinfo no encontró un módulo independiente, la configuración CONFIG_BLK_DEV_LOOP=y confirmó que su soporte estaba integrado en el kernel. Posteriormente, mediante losetup se asoció un archivo temporal con /dev/loop0 y se realizó una lectura exitosa utilizando dd. Por lo tanto, se comprobó su funcionamiento para las operaciones evaluadas. En cambio, aunque el kernel incluía soporte modular para KVM, no se encontraron módulos cargados, la extensión svm no aparecía expuesta y /dev/kvm estaba ausente. Estos resultados permitieron distinguir entre soporte configurado, disponibilidad de recursos y funcionamiento efectivo, sin generalizar las condiciones observadas a todas las instalaciones de WSL2.

En relación con los dispositivos físicos, Windows reconoció el dispositivo de almacenamiento USB, mientras que Ubuntu no lo enumeró inicialmente. Mediante usbipd-win se logró conectar el dispositivo a WSL2 y verificar posteriormente su desconexión. Esto permitió comprobar que el acceso a determinados dispositivos USB requiere mecanismos adicionales de integración con el sistema anfitrión, sin representar una incompatibilidad absoluta con estos dispositivos.

Finalmente, la prueba de persistencia mostró que Nginx permanecía habilitado y activo mediante systemd, pero la distribución Ubuntu pasó de Running a Stopped después de cerrar la última sesión interactiva. Este resultado evidencia que la administración interna de servicios Linux no garantiza, por sí sola, la permanencia de una instancia WSL2.

En conjunto, los resultados indican que WSL2 proporciona un entorno Linux funcional e integrado con Windows, aunque determinadas capacidades dependen de la configuración del kernel, de los recursos expuestos por la virtualización y de los mecanismos de integración con el sistema anfitrión. Las observaciones corresponden al equipo, las versiones y las condiciones experimentales utilizadas, por lo que no deben generalizarse a todas las configuraciones de WSL2.

5. Conclusión y posibles mejoras

La investigación permitió analizar el funcionamiento de Windows Subsystem for Linux 2 (WSL2) mediante cinco experimentos sobre su arquitectura, conectividad, sistemas de archivos, acceso al hardware y administración de servicios. Los resultados confirmaron que WSL2 proporciona un entorno Linux funcional mediante un kernel real ejecutado sobre una infraestructura de virtualización ligera administrada por Windows.

Las pruebas evidenciaron particularidades en su integración con Windows. La conectividad requirió configuraciones adicionales, mientras que el acceso a archivos presentó diferencias de rendimiento y permisos. Mediante la herramienta de diagnóstico desarrollada en Python y pruebas complementarias, se comprobó que loop estaba integrado en el kernel y funcionaba correctamente para las operaciones evaluadas. En cambio, aunque existía soporte modular para KVM, /dev/kvm no estaba disponible en el entorno estudiado. Asimismo, el acceso USB requirió usbipd-win y la habilitación de Nginx mediante systemd no garantizó la permanencia de la distribución.

En conjunto, los resultados muestran que WSL2 constituye una alternativa útil para ejecutar aplicaciones Linux desde Windows, aunque determinadas capacidades dependen de la configuración y de los mecanismos de integración. Las dificultades reportadas por la comunidad también evidencian la importancia de evaluar cada entorno antes de generalizar sus restricciones.

Como posibles mejoras, se propone ampliar las pruebas de rendimiento, investigar la disponibilidad de KVM en distintas configuraciones y profundizar el estudio del acceso a dispositivos físicos y la persistencia de servicios.

6. Referencias

[1] StatCounter GlobalStats, "Desktop Operating System Market Share Worldwide," septiembre de 2026. [En línea]. Disponible: https://gs.statcounter.com/os-market-share/desktop/worldwide. [Consulta: 27-oct-2026].

[2] W3Techs, "Usage of Linux as operating system broken down by ranking," octubre de 2026. [En línea]. Disponible: https://w3techs.com/technologies/breakdown/os-linux/ranking. [Consulta: 27-oct-2026].

[3] Microsoft, "¿Qué es el Subsistema de Windows para Linux?", Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/about. [Consulta: 19-oct-2026].

[4] C. Loewen, "Announcing WSL 2," Windows Command Line Blog, Microsoft, 6 de mayo de 2019. [En línea]. Disponible: https://devblogs.microsoft.com/commandline/announcing-wsl-2/. [Consulta: 26-oct-2026].

[5] Microsoft, "Comparación de versiones de WSL," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/compare-versions. [Consulta: 26-oct-2026].

[6] Microsoft, "Trabajar en sistemas de archivos Windows y Linux," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/filesystems. [Consulta: 19-oct-2026].

[7] Microsoft, "Configuración avanzada en WSL," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/wsl-config. [Consulta: 19-oct-2026].

[8] Microsoft, "Acceso a aplicaciones de red con WSL," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/networking. [Consulta: 26-oct-2026].

[9] Microsoft, "Conectar dispositivos USB," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/connect-usb. [Consulta: 29-oct-2026].

[10] Microsoft, "Usar systemd para administrar servicios de Linux con WSL," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/systemd. [Consulta: 29-oct-2026].

[11] Microsoft, "Comandos básicos para WSL," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/es-es/windows/wsl/basic-commands. [Consulta: 29-oct-2026].

[12] microsoft/WSL, «Loopback device partitions missing from /dev/loop*», GitHub Issues, n.º 11170, 2024. [En línea]. Disponible: https://github.com/microsoft/WSL/issues/11170. [Consulta: 01-nov-2026].

[13] microsoft/WSL, «nestedVirtualization silently disabled: HCS ProcessorCapabilities query fails with 0x8007000D on 26100.9711 (Oct 2026 CU), WSL swallows error», GitHub Issues, n.º 41917, octubre de 2026. [En línea]. Disponible: https://github.com/microsoft/WSL/issues/41917. [Consulta: 01-nov-2026].

[14] dorssel/usbipd-win, «Failed to attach device after enable mirrored mode networking», GitHub Issues, n.º 882, marzo de 2024. [En línea]. Disponible: https://github.com/dorssel/usbipd-win/issues/882. [Consulta: 01-nov-2026].

[15] Microsoft, «October 28, 2026—KB5127936 (OS Builds 26300.9818 and 26200.9818) Preview», Microsoft Support, 28 de octubre de 2026. [En línea]. Disponible: https://support.microsoft.com/en-au/servicing/os/windows-11/2026/10/kb5127936-windows-11-25h2-26h2-update. [Consulta: 01-nov-2026].